

Week5 学習ノート：ユーザー別データ管理・RLS

日付：2026-05-10

今日のテーマ

「ログインユーザー以外のデータも見えてしまう」バグを発見し、2段階で修正した。

1. **アプリ側**：データ取得・追加時にユーザーIDで絞り込む
2. **Supabase側**：RLS（Row Level Security）でサーバーでも守る

問題の発見

何が問題だったか？

loadItems 関数でデータを取得するとき、こう書いていた：

```
const { data } = await supabase.from('expenses').select('*');
```

select('*') はテーブルの**全データ**を取ってくる。

ログインしているユーザーで絞り込んでいないので、**他のユーザーのデータも丸見え**になっていた。

修正1：アプリ側（コード）

ポイント： getSession() でユーザーIDを取得する

Supabase には supabase.auth.getSession() という関数があり、今ログインしているユーザーの情報を取得できる。

戻り値の構造

```
{
  data: {
    session: {
      user: {
        id: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" ← これが欲しい！
      }
    }
  }
}
```

分解代入で取り出す

```
const { data: { session } } = await supabase.auth.getSession();
// session.user.id でユーザーIDが取れる
```

豆知識： { data: { session } } はネストした分解代入といい、オブジェクトの中のオブジェクトを一度に取り出せる書き方。

データ取得の修正

修正前

```
const loadItems = async () => {
  const { data } = await supabase.from('expenses').select('*');
  // 全ユーザーのデータが取れてしまう
};
```

修正後

```
const loadItems = async () => {
  const { data: { session } } = await supabase.auth.getSession();
  const { data } = await supabase.from('expenses')
    .select('*')
    .eq('user_id', session.user.id); // 自分のデータだけ取得！
};
```

ポイント：

- `select('*')` は**取得する列**を指定するメソッド（絞り込みではない）
- `.eq('列名', 値)` で**行を絞り込む**（equal = 等しい）
- `getSession()` は `async` 関数なので `await` が必要

データ追加の修正

修正前

```
await supabase.from('expenses').insert({
  name: inputText,
  amount: Number(inputNumber),
  category: category,
  date: ...
});
// user_id が保存されていない！
```

修正後

```
const { data: { session } } = await supabase.auth.getSession();
await supabase.from('expenses').insert({
  user_id: session.user.id, // 追加！
  name: inputText,
  amount: Number(inputNumber),
  category: category,
  date: ...
});
```

修正2：Supabase側（RLS）

なぜコードだけでは不十分か？

コードで絞り込んでも、APIキーを知っている人がSupabaseに直接アクセスすると全データが見えてしまう。

RLS (Row Level Security) を使うと、サーバー側でも「自分のデータしか触れない」ルールを強制できる。

手順

① user_id 列を追加する

expenses テーブルに user_id (型: uuid、Allow Nullable: ON) を追加した。
※ 既存データに user_id がないので Nullable にする必要がある。

② RLSを有効にする

Authentication → Policies → expenses テーブルの **Enable RLS** をクリック。

③ ポリシーを3つ作成する

ポリシー	内容
SELECT	自分の user_id のデータだけ取得できる
INSERT	自分の user_id のデータだけ追加できる
DELETE	自分の user_id のデータだけ削除できる

SELECTポリシーの**USING**式：

```
auth.uid() = user_id
```

auth.uid() はSupabaseが用意している関数で、「今ログインしているユーザーのUUID」を返す。

今日のつまずきポイント

つまずき	正しい書き方
getSession() に await を付け忘れた	await supabase.auth.getSession()
select('U.user_id') で絞り込もうとした	select() は列指定、絞り込みは .eq()
user.'id' とクォートを付けてしまった	session.user.id (クォート不要)

つまり	正しい書き方
U を関数として定義してしまった	loadItems の中で直接 await して使う

今日学んだ重要な概念

分解代入（復習）

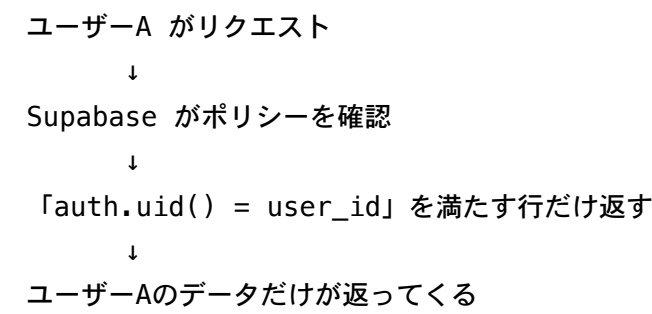
```
// 通常
const result = await supabase.auth.getSession();
const session = result.data.session;

// 分解代入（スッキリ！）
const { data: { session } } = await supabase.auth.getSession();
```

select() vs eq()

```
.select('*')           // どの列を取得するか（全列）
.select('id, name')    // id と name だけ取得
.eq('user_id', id)     // user_id が id と等しい行だけ取得
```

RLS の仕組み



まとめ

- セキュリティは**アプリ側**と**サーバー側**の両方で守る
- `getSession()` で現在のユーザーを取得し、 `session.user.id` でIDを使う
- Supabaseの RLS ポリシーで SELECT / INSERT / DELETE を制御する